Exercise Sheet 3: Least squares

○ Self-study † Class discussion

Class discussion: Questions 2, 3, 5.

Use the supplied housing.csv for Question 6. Keep it beside the notebook. Use Python 3 with NumPy, SciPy and Matplotlib. No assignment files are required.

Question 1 - Shorthand operators ○

For NumPy arrays:

Operator	Function form	Meaning
`A @ B`	`np.matmul(A, B)`	Matrix multiplication
`A * B`	`np.multiply(A, B)`	Elementwise multiplication

With one-dimensional vectors, `@` gives a dot product; a matrix and a vector give a matrix-vector product. Matching inner dimensions must agree. Use `*` for scalar multiplication too. Elementwise products can broadcast compatible unequal shapes.

Use these arrays; no hand calculations are required:

```
v = np.array([1., 2., 3.])
w = np.array([4., 5., 6.])
A = np.array([[1., 2., 3.],
              [4., 5., 6.]])
B = np.array([[1., 0.],
              [0., 1.],
              [1., 1.]])
C = np.array([[2., 1., 0.],
              [1., 0., 2.]])
```

(a) Run `v @ w`, `A @ v` and `A @ B`. Inspect each result and its shape: which is a scalar, a vector or a matrix? Compare each result with `np.matmul` and `np.dot`, using the same inputs and `np.allclose`. For these one- and two-dimensional examples, the three forms compute the same product.

(b) Run `v * w` and `A * C`. Compare each with `np.multiply` using `np.allclose`. Then compare `A * v` with `A @ v`: do they have the same shape or meaning? In `A * v`, NumPy broadcasts the three entries of `v` across each row of `A`, then multiplies entry by entry. Also run `2 * v` and `2 * A`.

(c) Run these expressions one at a time: `A @ A`, `A * B`, `v * w[:2]` and `v @ 2`. They are deliberately invalid. Read each error message and explain which dimension or operand rule is violated. Catch `ValueError` or `TypeError` if needed so that you can continue to the next example. Why does changing `@` to `*` not automatically fix a shape error?

(d) `np.allclose(a, b)` returns `True` if every pair of corresponding finite entries satisfies $|a_i - b_i| \leq \text{atol} + \text{rtol}|b_i|$. The defaults are `rtol=1e-5` and `atol=1e-8`: relative tolerance scales with `b`, while absolute tolerance supplies a fixed allowance near zero. Run this example with the defaults written explicitly:

```
a = np.array([1.000001, 0.000000005])
b = np.array([1.0, 0.0])
print(np.allclose(a, b, rtol=1e-5, atol=1e-8))
```

Keep the arrays fixed. First tighten only `rtol` to `1e-7`; then restore `rtol=1e-5` and tighten only `atol` to `1e-10`. Finally use `rtol=0, atol=1e-5`. Record each Boolean result and identify which entry passes or fails the tolerance condition. Why does `rtol` have no effect on the comparison with a reference value of zero?

Question 2 - Asymptotic notation †

Asymptotic notation compares how functions grow as n tends to infinity. It need not describe a program. Let n range over the positive integers and let $g(n) > 0$ for all sufficiently large n.

We write $f(n) = O(g(n))$ if there are constants $C > 0$ and n_0 such that $|f(n)| \leq Cg(n)$ for every $n \geq n_0$.

We write $f(n) = o(g(n))$ if, for every $\varepsilon > 0$, there is an n_0 such that $|f(n)| \leq \varepsilon g(n)$ for every $n \geq n_0$. Equivalently, $f(n)/g(n) \to 0$. Big-O gives an eventual bound up to a constant; little-o means the ratio tends to zero. Constants and thresholds must not depend on n; in the little-o definition, the threshold may depend on ε.

(a) Use the definition to show that $3n^2 + 5n + 7 = O(n^2)$ and $4n^3 - 2n + 1 = O(n^3)$. Give explicit values of C and n_0 for each. Is the quadratic also $o(n^2)$? Is the cubic $o(n^3)$ or $o(n^4)$? Justify your answers.

(b) Suppose $f = O(g)$ and $h = O(k)$, where g and k are eventually positive. Prove that $f + h = O(g+k)$ and $fh = O(gk)$. Deduce that $O(n)+O(n^2)$ terms together give an $O(n^2)$ bound, while their product gives an $O(n^3)$ bound. This shorthand describes bounds on functions; it is not arithmetic with fixed numbers named $O(n)$.

(c) If $f = o(g)$ and $h = O(g)$, must $f + h = o(g)$? Give a counterexample if not. If both $f = o(g)$ and $h = o(g)$, prove that $f + h = o(g)$.

(d) Suppose you know only that $f(n) = O(n^2)$. Can you conclude that $f(n)$ is not $O(n)$? Justify your answer with an explicit example. Explain why a valid big-O upper bound need not be the tightest possible bound.

Question 3 - Temperature and reaction rate †

A reaction is measured at ten absolute temperatures. The following positive rates are illustrative measurements, not a real experimental dataset. Temperature T is in kelvin; the rate r is recorded numerically in one fixed unit.

```
T = np.array([280., 290., 300., 310., 320.,
              330., 340., 350., 360., 370.])
r = np.array([0.39, 0.50, 0.66, 0.88, 1.12,
              1.40, 1.70, 2.13, 2.56, 3.08])
```

A proposed model is $r(T) = C\exp(-B/T)$, with unknown positive constants C and B. Increasing temperature increases the predicted rate. You do not need chemistry to answer the question.

(a) Take natural logarithms and rearrange the model into a straight-line relationship. What transformed input and response should you use? Identify how the line's intercept and slope relate to C and B. Explain why the temperatures and measured rates must be positive.

(b) Construct a design matrix with an intercept column. Fit the transformed observations using `np.linalg.lstsq(A, y, rcond=None)`. Recover estimates of C and B from the returned coefficients. State the units of B needed to make B/T dimensionless. We take logarithms of the numerical rates expressed in the fixed unit above.

(c) Create two panels. In the first, show the transformed data points together with the fitted straight line. In the second, show all ten original measurements as a scatter plot and overlay the fitted function $\hat{r}(T) = \hat{C}\exp(-\hat{B}/T)$ as a smooth curve on the same axes. Use `np.linspace(280, 370, 200)` to draw the curve, and add axis labels and a legend distinguishing measurements from the fitted model. How well does the curve follow the observations? Predict the rate at 335 K, which is not one of the measured temperatures. Is that interpolation or extrapolation?

(d) Compute the root sum of squared errors in the original rate units: $E = \sqrt{\sum_i (r_i - \hat{r}(T_i))^2}$. Also compute the sum of squared errors in the transformed response. Which error criterion did your fitting procedure minimize? Are the two error values in the same units?

Question 4 - Solving without an inverse ◦

Although $x = A^{-1}b$ is a mathematical formula for the solution of a nonsingular square system, computing the inverse is usually unnecessary. Compare the two computational routes below.

(a) Solve $2u+v = 5$, $u+3v+w = 10$ and $v+2w = 5$. Write $Ax = b$, create floating-point arrays, and use `np.linalg.solve(A, b)`. Check the equations with `np.allclose(A @ x, b)`. Then calculate `np.linalg.inv(A) @ b`. Do the answers agree for this small example?

(b) Compare the time taken on this larger, reproducible system:

```
rng = np.random.default_rng(587)
n = 300
M = rng.normal(size=(n, n))
A = M.T @ M + np.eye(n)
x_ref = np.ones(n)
b = A @ x_ref
```

The added identity makes A positive definite and hence nonsingular. Time `np.linalg.solve(A, b)` and `np.linalg.inv(A) @ b` using `timeit.repeat`, with `number=3` and `repeat=3`. Report the minimum batch time divided by 3 as seconds per solution. Include computing the inverse on every call; prepare the data and print results outside the timed calculation. Which route is faster? Both have $O(n^3)$ leading work for a dense square matrix; the same big-O order does not mean equal running time. The inverse route also constructs an entire n-by-n result when only one solution vector is needed.

(c) Now compare accuracy on a sensitive system. Use this Hilbert matrix, whose entries are $H_{ij} = 1/(i+j+1)$ with indices starting at zero:

```
n = 10
i = np.arange(n)
H = 1.0 / (i[:, None] + i[None, :] + 1)
x_ref = np.ones(n)
b = H @ x_ref
x_solve = np.linalg.solve(H, b)
x_inverse = np.linalg.inv(H) @ b
```

This matrix is nonsingular mathematically, but ill-conditioned: small rounding or data changes can cause large changes in the solution. For each computed vector x, report the relative residual $\|b-Hx\|_2/\|b\|_2$ and the relative error against the reference $\|x-x_{\text{ref}}\|_2/\|x_{\text{ref}}\|_2$, using `np.linalg.norm`. Try sizes 8, 10 and 12. Does the inverse route leave a larger residual? Can a tiny residual coexist with a noticeable solution error? Results vary with the numerical library and machine; neither method is guaranteed to retain high accuracy on an ill-conditioned problem. Even constructing b introduces rounding.

(d) Based on the time, storage and error comparisons, which method would you choose when you need only the solution of $Ax = b$? Explain why agreement on the small example in part (a) is not sufficient reason to form an inverse in general. You do not need to repeat the timings for the Hilbert matrices.

Question 5 - QR and least squares †

Use the following fixed matrix and observations throughout this question:

```
A = np.array([[1., 0., 0.],
              [1., 1., 1.],
              [1., 2., 0.],
              [1., 3., 1.],
              [1., 4., 0.],
              [1., 5., 1.]])
y = np.array([1., 2., 2., 4., 4., 5.])
```

The columns represent an intercept, a numerical input and a binary indicator. We want coefficients θ that minimize $\|y - A\theta\|_2^2$.

(a) Check that the columns are independent using `np.linalg.matrix_rank(A)`. Find the least-squares coefficients in two ways: reduced QR followed by `scipy.linalg.solve_triangular(R, Q.T @ y)`, and `np.linalg.lstsq(A, y, rcond=None)[0]`. Compare the coefficients with `np.allclose`, and compare the residual norms $\|y - A\theta\|_2$. What are the shapes of Q and R?

(b) Put the complete QR calculation in a function `qr_fit(A, y)`. It must compute Q and R, form $Q^T y$, and solve the triangular system on every call. With the arrays `A` and `y` already defined, run:

```
import timeit
qr_times = timeit.repeat(lambda: qr_fit(A, y),
                         number=100, repeat=3)
ls_times = timeit.repeat(
    lambda: np.linalg.lstsq(A, y, rcond=None),
    number=100, repeat=3)

print("QR seconds per fit:", min(qr_times) / 100)
print("lstsq seconds per fit:", min(ls_times) / 100)
```

Here `lambda: ...` tells the timer which calculation to run. Each method is run 100 times in a batch, and three batches are measured. Taking the fastest batch and dividing by 100 gives an average time per fit for that batch. Creating `A` and `y`, and printing the results, are not included in these measured times. Which method takes less time for this matrix? The answer describes this experiment; the faster method may differ for a larger matrix.

(c) Now make the third column twice the second:

```
A_dep = A.copy()
A_dep[:, 2] = 2 * A_dep[:, 1]
```

Keep the same y. Check the rank and repeat both fitting methods. Inspect the diagonal of R, the coefficients and the residual norms. Catch `np.linalg.LinAlgError` if the triangular solve fails. It may instead return large coefficients because rounding stores a tiny diagonal entry rather than exact zero. Explain why the coefficients are no longer unique, even if a small residual is obtained. What choice does `lstsq` make among the minimizing coefficient vectors? You do not need to repeat the timing experiment for this case.

Question 6 - Polynomial models on housing data ◦

Use the supplied `housing.csv`, containing 33 measurements of a US housing price index. Each row contains an elapsed time in years and the corresponding price index. Treat this as a curve-fitting exercise: how much polynomial flexibility is useful for describing the relationship? We are not forecasting future prices.

(a) Open `housing.csv` in a text editor and inspect the header and first few rows. Identify the two columns. Load the numerical values into a two-dimensional NumPy array using `np.loadtxt` with `delimiter=","` and `skiprows=1`. Why is the header skipped? Print the array shape and first three rows, then save it as `housing.npy` using `np.save`. Reload the saved file with `np.load` and use `np.array_equal` to check that the numerical values have been preserved.

(b) Use the `housing.npy` you created to make a scatter plot with labelled axes:

```
data = np.load("housing.npy")
t = data[:, 0]
y = data[:, 1]
x = t / 16.0
```

Here `t` is the number of years since the first observation: 0 is the first observation and 16 is sixteen years later. Plot `t` on the horizontal axis and the price index `y` on the vertical axis. Does the pattern look roughly straight, or does it bend in a way a straight line would miss? For the polynomial calculations below, use `x = t / 16.0`, which maps 0 years to 0 and 16 years to 1. This avoids taking high powers of larger input values. Keep `y` unchanged, and use this same input scaling when fitting and predicting.

(c) Use one fixed shuffled split for every model:

```
rng = np.random.default_rng(587)  # Fixed seed: reproducible split
indices = rng.permutation(len(y))
train = indices[:21]
validation = indices[21:27]
test = indices[27:]
```

These are row indices; for example, `x[train]` selects the training inputs. State what the three sets are used for. Keep the test responses out of fitting and degree selection. This shuffled split assesses curve fitting within the observed range, not performance as a future-price forecast.

(d) Fit polynomial degrees $d = 1, 2, 3, 5, 8$ using only the training observations. Create arrays for the columns $1, x, x^2, \ldots, x^d$ and stack them using `np.column_stack` to form the design matrix. The first column must contain ones. Use the training inputs to build the fitting matrix, then obtain coefficients using `np.linalg.lstsq`. Calculate training and validation RMSE for each degree: $\text{RMSE} = \sqrt{\frac{1}{m}\sum_{i=1}^{m}(y_i - \hat{y}_i)^2}$. Build prediction matrices with the same column order. Record the errors in a table.

(e) In a 2-by-3 subplot figure (leave the sixth panel blank), show each fitted polynomial as a smooth curve over the full input range, together with the training and validation points in different colours. Draw curves on a sorted, fine grid, not in the shuffled row order. Label the horizontal axis in elapsed years; convert grid values to the scaled input when constructing the prediction matrix. Use matching axes to make the fits comparable. Which degrees miss important structure, and which introduce unnecessary bends? Plot training and validation RMSE against degree as well.

(f) Choose a degree using validation error and the plots. If several degrees perform similarly, explain whether you prefer the simpler model. Why should training error alone not determine the degree? Fix your choice, refit it on the 27 training-plus-validation observations, and evaluate the six test observations once. Compare its test RMSE with a constant prediction equal to the mean training-plus-validation price index. Finally, plot the selected curve with all observations, distinguishing the test points. Do not change the degree after seeing the test error.